

Cooperative Multi-Agent Reinforcement Learning with Technical Analysis for Bitcoin Trading: A Q-Learning Approach

Melika Dehghani
dept. Computer Engineering
Islamic Azad University, Science and
Research Branch
Tehran, Iran
dehghani.melika@gmail.com

MohammadHosein Panahi
dept. Electrical and Computer
Engineering
University of Tehran
Tehran, Iran
hosein.panahi@ut.ac.ir

Pedram Motamedi
dept. Computer Engineering
Amirkabir University of Technology
(Tehran Polytechnic)
Tehran, Iran
p_motamedi@aut.ac.ir

Ehsan Panahi
dept. Electrical and Computer
Engineering
University of Tehran
Tehran, Iran
ehsan.panahi@ut.ac.ir

Abdorreza Hesam Mohseni
dept. Computer Engineering
University of Guilan
Rasht, Iran
abdorreza.mohseni@alum.sharif.edu

Abstract—This study presents a novel multi-agent reinforcement learning (MARL) framework for cryptocurrency trading, addressing the challenges of market volatility and dynamic price movements. By implementing a Q-learning approach within a Markov Decision Process framework, we develop specialized Buy and Sell Agents that collaborate to optimize trading decisions using the Engulfing technical indicator. Our approach overcomes the limitations of single-agent strategies by enabling more adaptive and sophisticated decision-making. Applied to Bitcoin (BTC/USD) trading data from March 2018 to March 2022, the framework outperforms traditional methods, including single-agent DDPG, LSTM-based strategies, and buy-and-hold approaches. The results show an 8% higher annual profit than the second-best strategy, a 57% win rate, and a Calmar ratio of 2.8, demonstrating superior profitability and risk management. This research advances computational trading strategies by highlighting the effectiveness of MARL in complex financial environments, offering insights for developing more intelligent and adaptive trading systems.

Keywords—multi-agent reinforcement learning, cryptocurrency trading, q-learning, technical analysis

I. INTRODUCTION

Financial markets, particularly cryptocurrency trading, have become central to the global economy, offering vast opportunities for investment and profit maximization. However, the volatility, complexity, and high-speed dynamics of cryptocurrency markets pose significant challenges for decision-making and strategy optimization. Traditional approaches, such as fundamental and technical analyses, often struggle to keep pace with these rapidly evolving environments, making them less effective in predicting trends or capturing opportunities for profit. As a result, there has been a growing interest in leveraging computational models and machine learning techniques, which provide adaptive, data-driven solutions for financial trading.

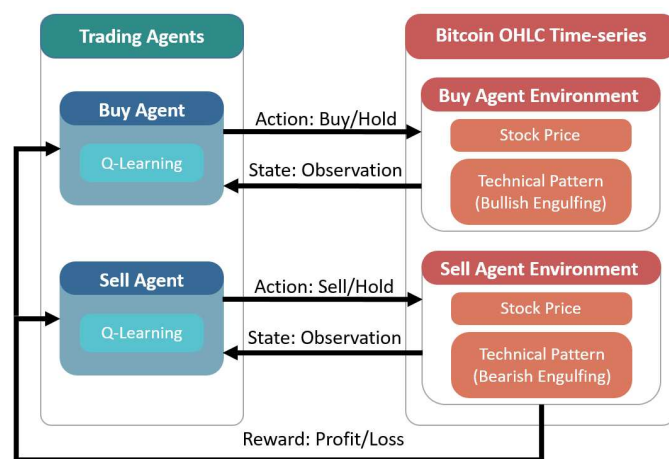


Fig. 1. Overview of the proposed reinforcement learning trading strategy

Among these approaches, deep learning models have gained prominence for their ability to extract patterns from large datasets. In particular, Long Short-Term Memory (LSTM) networks, a type of recurrent neural network, are well-suited for handling sequential data. Their capability to capture long-term dependencies makes them highly effective in processing financial time-series data, such as historical price trends in cryptocurrency trading. However, while LSTMs improve predictive accuracy, they do not inherently incorporate decision-making frameworks that optimize trading actions.

Reinforcement Learning (RL) offers a promising alternative by enabling autonomous agents to learn optimal behaviors through trial-and-error interactions with an environment. It has been successfully applied in various domains, including gaming, autonomous driving, and the Internet of Things (IoT) [1], demonstrating its potential to handle complex, dynamic decision-making tasks. In financial trading, RL agents aim to maximize cumulative rewards, which correspond to profits generated from successful trades.

Building on this foundation, we propose a novel multi-agent RL framework designed to enhance cryptocurrency trading strategies, as shown in Fig. 1. By leveraging Q-learning, the framework estimates the expected reward of actions in specific states, guiding agents to select decisions that yield the highest returns. This multi-agent system is structured within a Markov Decision Process (MDP) framework, modeling the trading environment in terms of states, actions, and rewards. The agents collaborate to refine trading strategies based on a well-known technical indicator, Engulfing [2], allowing for improved adaptability in volatile market conditions.

To evaluate its effectiveness, we applied the framework to a cryptocurrency trading dataset and compared its performance with traditional single-agent Deep Deterministic Policy Gradient (DDPG) and LSTM-based strategies. The results demonstrated that our multi-agent approach outperformed these baselines across several key trading performance indicators, including annual profit, maximum drawdown, win rate, and the Calmar ratio. Notably, it achieved 8% higher annual profit than the second-best approach, which was the single-agent DDPG strategy. These findings highlight the practical applicability of the proposed framework in maximizing returns while mitigating financial risks in cryptocurrency trading.

The remainder of this paper is organized as follows: Section II reviews related work in financial market analysis and algorithmic trading. Section III details the proposed RL methodology. Section IV presents experimental results on the cryptocurrency trading dataset, comparing the proposed model's performance to traditional approaches. Finally, Section V concludes the study, highlighting its implications and directions for future research in financial decision-making systems.

II. RELATED WORKS

Research on financial market prediction has explored various approaches, ranging from traditional technical analysis to advanced machine learning and reinforcement learning techniques. This section reviews relevant literature in three key areas: (1) studies that apply technical analysis to trading, (2) research utilizing deep learning for market prediction, and (3) reinforcement learning-based trading strategies.

A. Technical Trading

According to the Efficient Market Hypothesis (EMH), technical analysis should not generate abnormal returns, as it assumes that all available information is already incorporated into asset prices. This suggests that consistently outperforming the market using only technical indicators would be difficult [3]. However, numerous studies have examined the weak-form pricing efficiency of cryptocurrency assets, with most rejecting the null hypothesis of market efficiency [4][5][6]. This rejection implies that there may be opportunities to develop profitable trading strategies by leveraging information not yet fully reflected in market prices.

Expanding on this, [7] tested various technical trading rules, including moving averages and trading range breakout strategies, in the cryptocurrency market. Among these, the variable-length moving average rule proved to be the most effective, generating higher returns from buy signals compared to sell signals. Similarly, [8] analyzed seven

technical indicators using daily price data from July 2010 to January 2019 and found that trading range breakouts significantly predicted Bitcoin price movements, outperforming the buy-and-hold strategy, particularly in strongly trending markets. Additionally, [9] evaluated five technical indicators with multiple parameter configurations, resulting in 14,919 unique trading rules across the most liquid digital currency markets. Their findings underscored the profitability of technical trading strategies in these markets.

Building on these studies, further research has explored the integration of machine learning techniques into technical trading strategies. For instance, [10] utilized 128 indicators to predict the return range for the following day, employing a tree-based classifier and selecting the most successful strategy based on the win ratio. This strategy was then compared against the buy-and-hold approach. Likewise, [11] investigated the profitability of a deep learning model that incorporated technical indicators to forecast Bitcoin prices from 2016 to 2018. The study developed three distinct strategies, some of which generated positive excess returns, outperforming the buy-and-hold strategy. Lastly, [12] applied both hourly and daily data, integrating various technical indicators and non-linear models, and concluded that the random forest model was the most accurate predictor of Bitcoin prices.

B. Deep Learning in Trading

[13] evaluated the effectiveness of various classifiers, including feed-forward neural networks (FFNNs), for predicting stock price direction. Their framework leveraged historical stock data and compared the accuracy of multiple machine learning techniques. The model incorporated technical indicators such as moving averages and momentum indicators as input features. Extensive testing revealed that optimized FFNNs outperformed traditional methods like logistic regression and decision trees, demonstrating superior predictive accuracy. The results suggest that FFNNs have significant potential in stock price prediction and contribute to the advancement of trading strategies in the financial sector.

Building on this, [14] explored the application of LSTM networks for stock market trend prediction, addressing key limitations of Recurrent Neural Networks (RNNs), such as gradient vanishing and explosion. By incorporating input, forget, and output gates, LSTMs effectively mitigate these challenges, enabling them to retain crucial sequential information and enhance predictive performance. The study underscored LSTMs' ability to process long time-series data for forecasting stock movements, highlighting their potential as a powerful tool in financial market analysis.

Further extending LSTM applications, [15] employed LSTM networks to forecast stock returns in the Chinese market. Historical stock data was transformed into 30-day sequences as input for the LSTM model, which was trained using stochastic gradient descent with a learning rate of 0.001. The findings demonstrated that data normalization significantly improved prediction accuracy, increasing it from 15.6% to 20.1%. Additionally, incorporating the SSE index further enhanced accuracy to 24.1%. The study concluded that stock groupings influence prediction performance, emphasizing the importance of tailoring forecasting models for different stock categories.

Advancing deep learning techniques further, [16] implemented a Bidirectional Long Short-Term Memory

(BLSTM) network to enhance the learning process by utilizing both past and future input sequences. Unlike traditional unidirectional LSTM architectures, which process only past sequences stored in hidden layers, BLSTM networks capture information from both directions during training. Their findings showed that BLSTM networks outperformed unidirectional models, achieving superior accuracy and better convergence in both short-term and long-term stock price predictions.

C. Reinforcement Learning in Trading

Beyond LSTM, the integration of deep learning with Q-learning has been explored across various domains, leading to advancements in financial trading strategies. [17] introduced the deep Q-network (DQN), which combines Q-learning with deep convolutional neural networks (CNNs) to achieve impressive performance in tasks such as Atari games. While originally developed for gaming, this approach has since influenced financial trading applications, where reinforcement learning's ability to make sequential decisions offers significant advantages.

Expanding on this concept, [18] applied Deep Reinforcement Learning (DRL) to optimize stock trading strategies in complex and dynamic financial markets. The study utilized the DDPG algorithm, an advanced DRL method that integrates elements of Q-learning and policy gradient techniques. Operating within an actor-critic framework, the algorithm employs neural networks as function approximators—where the actor network learns the optimal trading strategy and the critic network evaluates the action-value function. This single-agent approach was tested on a selection of 30 stocks using daily price data for training. The results, benchmarked against the Dow Jones Industrial Average and a traditional mean-variance allocation strategy, demonstrated that the DDPG algorithm achieved higher returns and a superior Sharpe ratio, effectively balancing risk and reward in stock trading.

Further research has expanded the application of DRL to individual stock trading. [19] investigated multiple DRL methods, implementing algorithms such as Advantage Actor-Critic (A2C), Proximal Policy Optimization (PPO), DDPG, Twin Delayed DDPG (TD3), and Soft Actor-Critic (SAC) within a single-agent framework. These models were trained to make buy, sell, or hold decisions based on historical stock data, aiming to maximize trading profits. The study reinforced the potential of DRL-based approaches to adapt dynamically to market conditions and outperform traditional trading baselines.

Complementing these efforts, [20] proposed an automated stock trading strategy that integrates RL with a Gated Recurrent Unit (GRU) network. By leveraging the GRU network's ability to capture temporal dependencies in stock price movements, the RL agent was able to make more informed trading decisions. The integration of GRU enhanced the model's ability to process sequential data, improving predictive accuracy. When evaluated through backtesting on historical stock data, the strategy demonstrated promising results in terms of both profitability and risk management.

Laying an early foundation for multi-agent approaches, [21] explored the use of multi-agent Q-learning to optimize trading strategies. This study examined how multiple agents, each with an independent trading policy, could interact within a simulated financial environment to maximize cumulative

returns. Although conducted in an earlier period, it remains one of the few studies to directly apply multi-agent Q-learning to financial trading, serving as a basis for more recent advancements in the field.

III. METHOD

In this section, we describe the methodology used to address the multi-agent environment for trading tasks. The environment consists of two agents: a Buy Agent and a Sell Agent, each with its own set of tasks and interactions within the trading environment. The agents collaborate to optimize trading strategies through reinforcement learning.

A. Problem Definition

The objective of this work is to optimize trading strategies in a cryptocurrency market, specifically for Bitcoin, by employing a multi-agent reinforcement learning (MARL) framework. The problem is defined as a decision-making process where two agents, a Buy Agent and a Sell Agent, collaborate to maximize cumulative trading profits over a given time horizon. The environment is modeled using Bitcoin OHLC (Open, High, Low, Close) time-series data, augmented with technical patterns to guide the agents' decisions.

Each agent operates independently but is evaluated based on the combined profitability of their actions. The Buy Agent focuses on determining the optimal points to buy or hold, while the Sell Agent decides when to sell or hold. The agents interact with the environment, observe state transitions, and receive feedback in the form of rewards calculated based on trading performance. The overarching goal is to maximize the difference between selling and buying prices, thereby ensuring profitable trades.

B. MDP Definition

The task at hand is framed as a MDP, where both the Buy and Sell Agents aim to make decisions that maximize their expected long-term profits. The components of the MDP for this trading problem are as follows:

State Space (S): The state at each timestep is represented by observations that include the current stock price and any relevant technical patterns (e.g., Bullish Engulfing for the Buy Agent and Bearish Engulfing for the Sell Agent). Therefore, the state is defined by the pair of states $s_{\text{buy},t}$ and $s_{\text{sell},t}$, which correspond to the Buy Agent's and Sell Agent's perspectives of the environment at time t .

Action Space (A): The action space consists of two possible actions for each agent: "Buy" or "Hold" for the Buy Agent and "Sell" or "Hold" for the Sell Agent. This discrete action space is defined as:

$$A_{\text{buy}} = \{\text{Buy}, \text{Hold}\}$$

$$A_{\text{sell}} = \{\text{Sell}, \text{Hold}\}$$

Reward Function (R): The reward function provides the agents with feedback based on their actions. A reward of 1 is given when the Sell Agent's price exceeds the Buy Agent's price, representing a profitable trade, while a reward of -1 is assigned otherwise. This reward is used by both agents to update their policies. Thus, the reward is given as:

$$\text{Reward} = \begin{cases} 1 & \text{if sell_price} > \text{buy_price} \\ -1 & \text{otherwise.} \end{cases}$$

This design incentivizes agents to maximize the profit margin between buying and selling prices, with the Buy Agent focusing on purchasing at the optimal price and the Sell Agent aiming to sell at a higher price.

C. Q-learning Framework

Both the Buy and Sell Agents utilize Q-learning to iteratively refine their policies. The Q-function $Q(s, a)$ estimates the expected cumulative reward for taking action a in state s . Each episode begins with the initialization of the Q-functions $Q_{\text{buy}}(s_{\text{buy}}, a)$ and $Q_{\text{sell}}(s_{\text{sell}}, a)$, along with the respective policies π_{buy} and π_{sell} .

At each timestep t , the Buy Agent selects an action a_{buy} from its policy $\pi_{\text{buy}}(s_{\text{buy},t})$, while the Sell Agent selects an action a_{sell} from its policy $\pi_{\text{sell}}(s_{\text{sell},t})$. Following these actions, the next states $s_{\text{buy},t+1}$ and $s_{\text{sell},t+1}$ are observed, along with the corresponding rewards r_{buy} and r_{sell} .

The Q-function update for each agent is governed by the Bellman equation, which is used to update the Q-values iteratively based on the reward received and the maximum future reward. For the Buy Agent, the Q-value is updated as follows:

$$Q_{\text{buy}}(s_{\text{buy},t}, a_{\text{buy}}) \leftarrow Q_{\text{buy}}(s_{\text{buy},t}, a_{\text{buy}}) + \alpha \left(r_{\text{buy}} + \gamma \max_{a'} Q_{\text{buy}}(s_{\text{buy},t+1}, a') - Q_{\text{buy}}(s_{\text{buy},t}, a_{\text{buy}}) \right) \quad (1)$$

Similarly, for the Sell Agent, the Q-value is updated using the equation:

$$Q_{\text{sell}}(s_{\text{sell},t}, a_{\text{sell}}) \leftarrow Q_{\text{sell}}(s_{\text{sell},t}, a_{\text{sell}}) + \alpha \left(r_{\text{sell}} + \gamma \max_{a'} Q_{\text{sell}}(s_{\text{sell},t+1}, a') - Q_{\text{sell}}(s_{\text{sell},t}, a_{\text{sell}}) \right) \quad (2)$$

where α represents the learning rate, and γ is the discount factor.

The policy for each agent is updated by selecting the action that maximizes the Q-value for the given state. Thus, the policy for the Buy Agent is updated as:

$$\pi_{\text{buy}}(s_{\text{buy},t}) \leftarrow \arg\max_a Q_{\text{buy}}(s_{\text{buy},t}, a) \quad (3)$$

and for the Sell Agent, the policy update follows the equation:

$$\pi_{\text{sell}}(s_{\text{sell},t}) \leftarrow \arg\max_a Q_{\text{sell}}(s_{\text{sell},t}, a) \quad (4)$$

These equations ensure that both agents adjust their policies to maximize their respective expected cumulative rewards by learning from past interactions with the environment. The pseudocode for the proposed Q-learning approach is shown in Fig. 2.

Algorithm 1 Multi-Agent RL

```

Initialize Q-functions  $Q_{\text{buy}}(s_{\text{buy}}, a)$ ,  $Q_{\text{sell}}(s_{\text{sell}}, a)$  for both agents
Initialize Policies  $\pi_{\text{buy}}$ ,  $\pi_{\text{sell}}$  for both agents
repeat each episode
  Initialize  $s_{\text{buy},0}$ ,  $s_{\text{sell},0}$ 
  repeat each time step  $t$  do:
    Choose action  $a_{\text{buy}}$  from  $\pi_{\text{buy}}(s_{\text{buy},t})$  for Buy Agent
    Choose action  $a_{\text{sell}}$  from  $\pi_{\text{sell}}(s_{\text{sell},t})$  for Sell Agent
    Observe the next state  $s_{\text{buy},t+1}$ ,  $s_{\text{sell},t+1}$  and reward  $r_{\text{buy}}$ ,  $r_{\text{sell}}$ 

    Update Q-values:
    • Buy Agent :
      
$$Q_{\text{buy}}(s_{\text{buy},t}, a_{\text{buy}}) \leftarrow Q_{\text{buy}}(s_{\text{buy},t}, a_{\text{buy}}) + \alpha \times (r_{\text{buy}} + \gamma \times \max(Q_{\text{buy}}(s_{\text{buy},t+1}, a')) - Q_{\text{buy}}(s_{\text{buy},t}, a_{\text{buy}})) \quad (1)$$


    • Sell Agent :
      
$$Q_{\text{sell}}(s_{\text{sell},t}, a_{\text{sell}}) \leftarrow Q_{\text{sell}}(s_{\text{sell},t}, a_{\text{sell}}) + \alpha \times (r_{\text{sell}} + \gamma \times \max(Q_{\text{sell}}(s_{\text{sell},t+1}, a')) - Q_{\text{sell}}(s_{\text{sell},t}, a_{\text{sell}})) \quad (2)$$


    Update policies:
    • Buy Agent :  $\pi_{\text{buy}}(s_{\text{buy},t}) \leftarrow \arg\max_a Q_{\text{buy}}(s_{\text{buy},t}, a)$ 

    • Sell Agent :  $\pi_{\text{sell}}(s_{\text{sell},t}) \leftarrow \arg\max_a Q_{\text{sell}}(s_{\text{sell},t}, a)$ 

```

Fig. 2. Pseudocode of the multi-agent q-learning approach

D. Evaluation

We evaluate our trading strategies using four key performance metrics:

Annual Profit (Return): Represents the percentage return on investment over a one-year period, normalized for comparison. A higher annual return indicates better absolute performance. For example, an annual return of 14% means that an initial investment of \$100 would grow to \$114 over one year, excluding compound effects.

Win Rate: Represents the percentage of profitable trades relative to total trades executed. A win rate of 57% indicates that 57 out of 100 trades resulted in profit. While a higher win rate is generally better, it should be considered alongside the profit magnitude of winning versus losing trades.

Maximum Drawdown: Measures the largest decline from the initial investment value of \$1, expressed as a percentage. For example, a maximum drawdown of -5% means that at its lowest point, the portfolio declined to \$0.95 from the initial \$1 investment. This metric is crucial for understanding the strategy's worst historical decline from the initial investment value and assessing potential investment risks.

Calmar Ratio: A risk-adjusted performance measure calculated as the annual return divided by the maximum drawdown. It indicates how much annual return is generated per unit of maximum drawdown risk. A higher Calmar ratio suggests better risk-adjusted performance. For instance, our Multi-Agent Q-Learning's Calmar ratio of 2.8 means it generates 2.8 units of annual return per unit of maximum drawdown risk, indicating excellent risk-adjusted performance.

IV. EXPERIMENT

In this section, we evaluate the performance of our proposed MARL framework. We provide a detailed description of the dataset used, the experimental results, and a comparative analysis of our approach against several baseline methods.

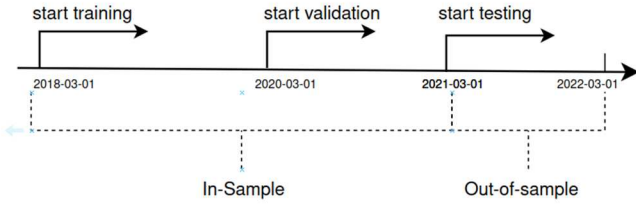


Fig. 3. Dataset split into training, validation, and testing samples dataset

We select Bitcoin (BTC/USD) as our trading asset for evaluating the proposed multi-agent reinforcement learning framework. Our backtesting uses historical daily OHLC data from 03/2018 to 03/2022 for performance evaluation. The cryptocurrency data is obtained from the Binance exchange through their public API. Our dataset consists of two periods: in-sample period and out-of-sample period as shown in Fig. 3. The in-sample period contains data for training and validation stages. The out-of-sample period contains data for the trading stage. In the training stage, we train our cooperative buy and sell agents using a multi-agent reinforcement learning approach. Then, a validation stage is carried out for validating the agents' performance and adjusting key parameters, such as learning rate, reward scaling, and network architectures. Finally, in the trading stage, we evaluate the profitability of our algorithm against the baseline methods.

The whole dataset is split as follows: data from 03/2018 to 03/2020 is used for training, and the data from 03/2020 to 03/2021 is used for validation and parameter tuning. Finally, we test our agents' performance on trading data, which is the unseen out-of-sample data from 03/2021 to 03/2022. To better exploit the trading data and adapt to market dynamics, we continue training our agents during the trading stage, allowing them to adjust to evolving market conditions while maintaining the core learned strategies from the training period.

A. Results and Analysis

The experimental results illustrated in Fig. 4 demonstrate the superior performance of our proposed MARL approach compared to all baseline strategies over the evaluation period from March 2021 to March 2022. Starting with an initial portfolio value of 1 USD for all strategies, our MARL framework executed a total of 80 complete trading positions, each consisting of a buy operation followed by a corresponding sell operation. The strategy achieved a final portfolio value of 1.14 USD, successfully outperforming the

Single Agent DDPG strategy (1.06 USD), the LSTM strategy (1.03 USD), and the buy-and-hold approach (0.66 USD).

Our MARL framework maintained a relatively consistent upward trend throughout the testing period, showing particular strength when compared to the high volatility displayed by the buy-and-hold strategy. This performance differential was most evident during the challenging period between May and July 2021, when the buy-and-hold approach experienced a substantial drawdown to approximately 0.51 USD, while our MARL strategy continued its upward trajectory. The framework's ability to identify profitable trading opportunities was particularly noteworthy during the upward trend between July and November 2021, where it captured significant gains through its active trading approach. The execution of 80 trading positions demonstrates the algorithm's capacity to maintain consistent market participation throughout the testing period.

Both the LSTM and Single Agent DDPG strategies demonstrated more stable price movements than the buy-and-hold approach, though each with distinct characteristics. The LSTM strategy showed notably conservative behavior, resulting in stable but moderate returns with a final portfolio value of 1.03 USD. The Single Agent DDPG, while more dynamic in its approach, achieved a final value of 1.06 USD, indicating that the single-agent architecture, though effective, couldn't match the sophisticated decision-making capabilities of our multi-agent system. Both approaches appeared less successful at capitalizing on market movements compared to our multi-agent system, which identified and acted upon more profitable trading opportunities.

During the market conditions after November 2021, our MARL strategy continued to maintain higher portfolio values compared to the baseline approaches. While other strategies showed varying degrees of performance decline during this period, our approach maintained its performance advantage through active trading. This sustained outperformance can be attributed to the cooperative nature of our multi-agent system, where the specialized buy and sell agents worked together to identify and execute profitable trades throughout the evaluation period.

The temporal evolution of portfolio values reveals distinct behavioral patterns among the four strategies. From the common starting point of 1 USD, our MARL framework showed consistent upward trajectory, while the buy-and-hold



Fig. 4. Portfolio value of trading strategies over time

TABLE I. RESULTS OF THE TRADING STRATEGIES

Strategy	Annual Profit (%)	Max Drawdown (%)	Win Rate (%)	Calmar Ratio
Multi-agent Q-learning	14	5	57	2.8
Single Agent DDPG	6	5	55	1.2
LSTM	3	8	51	0.37
Buy & Hold	-33	49	0	-0.6

strategy exhibited sharp fluctuations that ultimately led to significant value deterioration. The LSTM and Single Agent DDPG strategies, while more stable than buy-and-hold, failed to capture the full potential of market movements. The Single Agent DDPG's performance particularly highlights the advantages of our multi-agent approach, where the separation of buy and sell decisions into specialized agents appears to create more refined and effective trading strategies. The consistent execution of 80 trading positions throughout this period demonstrates our algorithm's ability to maintain active participation in the market while successfully growing the initial capital.

B. Comparative Analysis

The superior performance of our Multi-Agent Q-Learning framework can be attributed to several factors, as shown in Table 1. First, the significant gap in annual profit (14% vs 6% for the next best approach) demonstrates the advantage of specialized agents working cooperatively. The matched maximum drawdown with Single Agent DDPG (-5%) while achieving higher returns indicates that our approach maintains strong risk management without sacrificing performance.

The higher win rate (57%) suggests that the cooperation between buy and sell agents leads to more accurate trading decisions. This is particularly noteworthy given the complex nature of cryptocurrency markets. The exceptional Calmar Ratio of 2.8 provides strong evidence that our approach achieves superior risk-adjusted returns, effectively balancing profit generation with risk management.

These results collectively demonstrate that our Multi-Agent Q-Learning approach not only generates higher returns but does so with better risk management and more consistent performance compared to both traditional and advanced baseline methods.

V. CONCLUSION

This paper demonstrates the potential of MARL for optimizing cryptocurrency trading strategies by employing specialized Buy and Sell Agents within a Q-learning framework. Our experimental results show that the MARL approach outperforms traditional methods, including single-agent deep reinforcement learning, LSTM-based strategies, and the buy-and-hold approach, achieving a notable 14% annual profit with strong risk management. The key advantage lies in the cooperative agent design, which enhances decision-making by modeling the trading environment as a Markov Decision Process and incorporating technical patterns like the Engulfing indicator. The high win rate of 57% and an exceptional Calmar ratio of 2.8 further highlight the strategy's effectiveness in volatile markets. This research contributes to computational finance by demonstrating the benefits of multi-

agent architectures in trading, offering a promising direction for adaptive, data-driven strategies. Future work could explore additional technical indicators, broader asset classes, and more advanced multi-agent architectures, along with hybrid approaches that combine reinforcement learning with other machine learning techniques for further performance enhancements.

REFERENCES

- [1] Firouzi, Farshad, et al. "Task offloading for edge-fog-cloud interplay in the healthcare Internet of Things (IoT)." 2021 IEEE international conference on omni-layer intelligent systems (COINS). IEEE, 2021.
- [2] Achelis, Steven B. "Technical Analysis from A to Z." Apr. 2001.
- [3] Fama, Eugene F. "Efficient capital markets: A review of theory and empirical work." *The Journal of Finance* 25.2 (1970): 383-417.
- [4] Nadarajah, Saralees, and Jeffrey Chu. "On the inefficiency of Bitcoin." *Economics Letters* 150 (2017): 6-9.
- [5] Wei, Wang Chun. "Liquidity and market efficiency in cryptocurrencies." *Economics Letters* 168 (2018): 21-24.
- [6] Corbet, Shaen, et al. "The effectiveness of technical trading rules in cryptocurrency markets." *Finance Research Letters* 31 (2019): 32-37.
- [7] Gerritsen, Dirk F., et al. "The profitability of technical trading rules in the Bitcoin market." *Finance Research Letters* 34 (2020): 101263.
- [8] Hudson, Robert, and Andrew Urquhart. "Technical trading and cryptocurrencies." *Annals of Operations Research* 297.1-2 (2021): 191-220.
- [9] Huang, Jing-Zhi, William Huang, and Jun Ni. "Predicting bitcoin returns using high-dimensional technical indicators." *The Journal of Finance and Data Science* 5.3 (2019): 140-155.
- [10] Nakano, Masafumi, Akihiko Takahashi, and Soichiro Takahashi. "Bitcoin technical trading with artificial neural network." *Physica A: Statistical Mechanics and its Applications* 510 (2018): 587-609.
- [11] Gradojevic, Nikola, et al. "Forecasting Bitcoin with technical analysis: A not-so-random forest?" *International Journal of Forecasting* 39.1 (2023): 1-17.
- [12] Hutto, Clayton, and Eric Gilbert. "Vader: A parsimonious rule-based model for sentiment analysis of social media text." *Proceedings of the international AAAI conference on web and social media*. Vol. 8. No. 1. 2014.
- [13] Ballings, Michel, et al. "Evaluating multiple classifiers for stock price direction prediction." *Expert systems with Applications* 42.20 (2015): 7046-7056.
- [14] Wang, Jiajing, et al. "Predicting stock market trends using LSTM networks: overcoming RNN limitations for improved financial forecasting." *Journal of Computer Science and Software Applications* 4.3 (2024): 1-7.
- [15] Chen, Kai, Yi Zhou, and Fangyan Dai. "A LSTM-based method for stock returns prediction: A case study of China stock market." 2015 IEEE international conference on big data (big data). IEEE, 2015.
- [16] Althelaya, Khaled A., El-Sayed M. El-Alfy, and Salahadin Mohammed. "Evaluation of bidirectional LSTM for short-and long-term stock market prediction." 20189th international conference on information and communication systems (ICICS). IEEE, 2018.
- [17] Mnih, Volodymyr, et al. "Human-level control through deep reinforcement learning." *nature* 518.7540 (2015): 529-533.
- [18] Xiong, Zhuoran, et al. "Practical deep reinforcement learning approach for stock trading." *arXiv preprint arXiv:1811.07522* (2018).
- [19] Ge, Jun, et al. "Single stock trading with deep reinforcement learning: A comparative study." *Proceedings of the 2022 14th International Conference on Machine Learning and Computing*. 2022.
- [20] Yu, Jipu, et al. "Stock trading strategy based on reinforcement learning with GRU network." *Proceedings of the 2024 3rd International Conference on Cyber Security, Artificial Intelligence and Digital Economy*. 2024.
- [21] Lee, Jae Won, et al. "A multiagent approach to q q q -learning for daily stock trading." *IEEE Transactions on Systems, Man, and Cybernetics-Part A: Systems and Humans* 37.6 (2007): 864-87.